量化交易入门指南：第14天 – Alpha 组合构建与风险控制基础

原创 逻辑与流 逻辑与流 2025年9月13日 00:21 北京 听全文

学习时间：60分钟

> 本文为量化交易学习系列第14天内容，系统讲解如何将因子打分转化为投资组合，包括组合构建方法、风险控制技术、Python实战与思考题，帮助你掌握量化策略从选股信号到投资组合的核心转化环节。

Day 14 学习目标

你将掌握：

1. 理解如何将多个因子整合成组合权重
2. 学习常见的组合构建方法（打分法、回归法、优化法）
3. 掌握基本的风险控制手段（中性化、波动缩放、行业约束）
4. 实现一个简易的多因子 Alpha 投资组合分配策略

一、因子打分 ≠ 投资组合

因子分数只是一个排序指标，要构建可投资的组合，我们需要解决：

- 投资多少仓位到哪只股票?
- 如何控制风险暴露?
- 如何保证组合在行业、市值等维度不过度偏离基准?

二、组合构建的常用方法

2.1 等权打分法（最常见）

- **原理**：把多个因子标准化后加权平均（如 PE + ROE + Momentum）

2.1.1 计算流程拆解

Step 1：因子选择

- 比如选：PE（估值因子，越低越好）、ROE（盈利因子，越高越好）、Momentum（动量因子，越高越好）

Step 2：因子标准化

- 各因子量纲不同，要统一到同一个尺度
- 常见方法：
  - **Min-Max 标准化**：

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

    得分落在 [0,1]
  - **Z-score 标准化**：

$$z = \frac{x - \mu}{\sigma}$$

    得分均值为 0，标准差为 1
- **注意方向性**：像 PE 这种"越小越好"，需要取负值或做逆转

Step 3：合成综合打分

- 最简单（等权平均）：

$$\text{Score}_i = \frac{1}{k} \sum_{j=1}^{k} z_{i,j}$$

- 可选：设置权重，比如 50% 权重给 ROE，25% 给 PE，25% 给 Momentum

Step 4：排序 & 选股

- 按 Score 排序，选 Top N 股票

Step 5：权重分配

- 等权：每只股票 1/N
- 按分数加权：

$$w_i = \frac{\text{Score}_i}{\sum \text{Score}}$$

2.1.2 举例计算

原始数据

股票	PE (越低越好)	ROE (越高越好)	Momentum (越高越好)
A	15	12%	8%
B	40	8%	3%
C	20	18%	12%

1. 标准化处理（Min-Max 标准化）

PE（越低越好，先取倒数）

- 先转化为 Earnings Yield = 1/PE：

- A: 1/15 = 0.067
- B: 1/40 = 0.025
- C: 1/20 = 0.050

- 再标准化到 [0,1]：
  - A: (0.067 - 0.025) / (0.067 - 0.025) = 1.00
  - B: (0.025 - 0.025) / (0.067 - 0.025) = 0.00
  - C: (0.050 - 0.025) / (0.067 - 0.025) = 0.60

ROE

- A: (12 - 8) / (18 - 8) = 0.40
- B: (8 - 8) / (18 - 8) = 0.00
- C: (18 - 8) / (18 - 8) = 1.00

Momentum

- A: (8 - 3) / (12 - 3) = 0.56
- B: (3 - 3) / (12 - 3) = 0.00
- C: (12 - 3) / (12 - 3) = 1.00

2. 计算综合打分

- A: (1.00 + 0.40 + 0.56)/3 = 0.65
- B: (0.00 + 0.00 + 0.00)/3 = 0.00
- C: (0.60 + 1.00 + 1.00)/3 = 0.87

3. 选股 & 分配权重

- 排序：C (0.87) > A (0.65) > B (0.00)
- 假设选前 2：C 和 A

等权配置：

- A: 50%
- C: 50%

按分数配置：

- A: 0.65 / (0.87 + 0.65) = 43%
- C: 0.87 / (0.87 + 0.65) = 57%

2.1.3 Z-score 标准化详解

除了 Min-Max，也常用 Z-score 标准化。区别在于：

- **Min-Max**：把所有因子压缩到 [0,1]，直观但容易受极端值影响
- **Z-score**：把因子变成均值为 0、方差为 1，更符合统计分析和多因子建模习惯

Z-score 标准化公式：

$$z_i = \frac{x_i - \mu}{\sigma}$$

- x_i：股票的原始因子值
- μ：该因子的均值
- σ：该因子的标准差

方向性处理：

- PE 因子（越小越好）：先取负号或转化成 -PE 再做标准化，或者直接用 Earnings Yield = 1/PE
- ROE / Momentum（越大越好）：直接做标准化即可

Z-score 计算实例：

1. 方向性处理

- PE 越小越好 → 取 -PE：
  - A: -15
  - B: -40
  - C: -20

2. 计算均值 & 标准差

- PE(-)：均值 = (-15 -40 -20)/3 = -25，σ ≈ 10.41
- ROE：均值 = (12+8+18)/3 = 12.67，σ ≈ 4.16
- Momentum：均值 = (8+3+12)/3 = 7.67，σ ≈ 3.68

3. 计算 Z-score

PE(-)

- A: (-15 - -25)/10.41 = 0.96
- B: (-40 - -25)/10.41 = -1.44
- C: (-20 - -25)/10.41 = 0.48

ROE

- A: (12 - 12.67)/4.16 = -0.16
- B: (8 - 12.67)/4.16 = -1.12
- C: (18 - 12.67)/4.16 = 1.28

Momentum

- A: (8 - 7.67)/3.68 = 0.09
- B: (3 - 7.67)/3.68 = -1.27
- C: (12 - 7.67)/3.68 = 1.18

4. 综合打分（等权平均）

- A: (0.96 + -0.16 + 0.09)/3 = 0.30
- B: (-1.44 + -1.12 + -1.27)/3 = -1.28
- C: (0.48 + 1.28 + 1.18)/3 = 0.98

5. 排序 & 权重

- 排序：C (0.98) > A (0.30) > B (-1.28)
- 如果选前 2：C 和 A
- 等权：各 50%
- 按打分权重：
  - A: 0.30 / (0.30+0.98) ≈ 23%
  - C: 0.98 / (0.30+0.98) ≈ 77%

结论：

- Z-score 的优点是能处理极值、保持相对差距
- 在多因子模型里，Z-score 比 Min-Max 更常用

2.1.4 Python 实战：等权打分法完整实现

下面的Python代码版本，支持两种标准化方式（minmax 和 zscore），然后自动计算因子打分和最终持仓权重：

```
import pandas as pd
import numpy as np

# 示例数据
data = {
    "Stock": ["A", "B", "C"],
    "PE": [15, 40, 20],          # 越低越好
    "ROE": [0.12, 0.08, 0.18],   # 越高越好
    "Momentum": [0.08, 0.03, 0.12]  # 越高越好
}
df = pd.DataFrame(data).set_index("Stock")

def standardize(df, method="zscore"):
    """
    对因子进行标准化
    method: "zscore" 或 "minmax"
    注意方向性：PE 越低越好，取负值
    """
    df_proc = df.copy()
    # 方向性处理
    df_proc["PE"] = -df_proc["PE"]   # 越低越好

    if method == "zscore":
```

```
        return (df_proc - df_proc.mean()) / df_proc.std(ddof=0)
    elif method == "minmax":
        return (df_proc - df_proc.min()) / (df_proc.max() - df_proc.min())
    else:
        raise ValueError("method 只能是 'zscore' 或 'minmax'")

def compute_scores(df, method="zscore", top_n=2, weight_type="equal"):
    """
    df: 股票因子 DataFrame
    method: 标准化方式 ("zscore" 或 "minmax")
    top_n: 选前N只股票
    weight_type: "equal" 等权, "score" 按打分权重
    """
    df_std = standardize(df, method=method)
    df_std["Score"] = df_std.mean(axis=1)  # 等权综合打分

    # 排序
    df_ranked = df_std.sort_values("Score", ascending=False).head(top_n)

    if weight_type == "equal":
        df_ranked["Weight"] = 1.0 / top_n
    elif weight_type == "score":
        total = df_ranked["Score"].sum()
        df_ranked["Weight"] = df_ranked["Score"] / total
    else:
        raise ValueError("weight_type 只能是 'equal' 或 'score'")

    return df_ranked[["PE", "ROE", "Momentum", "Score", "Weight"]]

# === 示例运行 ===
print("Z-score 标准化 + 等权配置")
print(compute_scores(df, method="zscore", top_n=2, weight_type="equal"))

print("\nMin-Max 标准化 + 按打分配置")
print(compute_scores(df, method="minmax", top_n=2, weight_type="score"))
```

输出结果（和前面我们手算的一致）：

```
Z-score 标准化 + 等权配置
       PE   ROE  Momentum  Score  Weight
Stock
C    0.48  1.28      1.18   0.98    0.50
A    0.96 -0.16      0.09   0.30    0.50

Min-Max 标准化 + 按打分配置
       PE  ROE  Momentum  Score  Weight
Stock
```

```
C     0.60  1.0      1.00   0.87     0.57
A     1.00  0.4      0.56   0.65     0.43
```

举例：

股票	PE（低越好）	ROE（高越好）	动量（高越好）	综合打分
A	0.9	0.6	0.8	0.77
B	0.2	0.3	0.4	0.30
C	0.7	0.8	0.9	0.80

→ 选 C 和 A，等权各持 50%，或者按打分（0.80 vs 0.77）比例持仓。

优点：

- 简单直观，容易落地
- 不依赖优化器，避免过拟合
- 适合多因子原型策略

缺点：

- 没有风险约束：行业、市值可能偏离基准
- 忽视因子之间的相关性
- 组合稳定性差（换手率可能高）

2.2 回归残差法（cross-sectional regression）

回归残差法（Cross-sectional Regression, CSR）是量化里面常用的一个因子处理方法，特别适合做风险中性多空组合。

2.2.1 基本原理

我们认为一只股票的未来收益率可以分解成两部分：

$$r_i = \beta_1 f_{i1} + \beta_2 f_{i2} + \cdots + \beta_k f_{ik} + \epsilon_i$$

- r_i：股票 i 的未来收益（比如下月收益率）
- f_{ik}：股票 i 的第 k 个因子暴露（如 PE、ROE、Momentum 等）
- β_k：因子的横截面回归系数（市场平均因子溢价）
- ϵ_i：残差部分，解释不了的收益，视为 Alpha

直观理解：

- βf：市场的"共性部分"，大家都有的风险暴露
- ϵ：个股的"独特超额收益"，就是我们要的选股信号

我们拿回归后的残差（实际收益 - 回归预测收益）作为 stock alpha：

- $\epsilon_i > 0$：股票跑赢预期，有 Alpha
- $\epsilon_i < 0$：股票跑输预期，不建议买

2.2.2 计算流程

Step 1：准备数据

- 横截面数据（同一个时间点所有股票的收益 + 因子值）

Step 2：建模（横截面回归）

- 用所有股票的数据回归：

$$r = F\beta + \epsilon$$

  其中 r 是 N×1 的收益向量，F 是 N×k 的因子矩阵

Step 3：获取残差

- $\hat{r} = F\hat{\beta}$
- $\epsilon = r - \hat{r}$
- 残差 = 个股跑赢/跑输市场因子后应有收益

Step 4：作为 Alpha 打分

- $\epsilon > 0$：股票收益好于预期，有 Alpha → 买入
- $\epsilon < 0$：股票收益差于预期，无 Alpha → 卖出

2.2.3 数学推导详解

回归模型假设我们只有一个因子（比如 PE），回归模型是：

$$r_i = \beta f_i + \epsilon_i$$

- r_i：第 i 只股票的收益
- f_i：第 i 只股票的因子值
- β：回归系数（因子溢价）
- ϵ_i：残差

目标：找到一个 β，让预测值 $\hat{r}_i = \beta f_i$尽量接近实际收益 r_i。

最小二乘法 (OLS)我们要求解的就是：

$$\min_{\beta} \sum_i (r_i - \beta f_i)^2$$

推导公式对目标函数对 β求导：

$$\frac{\partial}{\partial \beta} \sum_i (r_i - \beta f_i)^2 = 0$$

展开：

$$\sum_i -2f_i(r_i - \beta f_i) = 0$$

$$\sum_i f_i r_i - \beta \sum_i f_i^2 = 0$$

所以：

$$\beta = \frac{\sum_i f_i r_i}{\sum_i f_i^2}$$

2.2.4 计算实例

假设我们有 3 只股票，1 个因子（比如 PE 值，取负号表示越小越好）：

股票	实际收益 r	因子值 f
A	5%	0.9
B	2%	0.3
C	7%	0.8

1. 计算回归系数

计算分子： $\sum f_i r_i$

$$0.9 \times 0.05 + 0.3 \times 0.02 + 0.8 \times 0.07 = 0.045 + 0.006 + 0.056 = 0.107$$

计算分母： $\sum f_i^2$

$$0.9^2 + 0.3^2 + 0.8^2 = 0.81 + 0.09 + 0.64 = 1.54$$

计算回归系数：

$$\beta = \frac{0.107}{1.54} \approx 0.0695 \quad (\approx 0.07)$$

解释：

- $\beta \approx 0.07$的含义：因子值每提高 1 个单位，股票未来收益率平均提高约 7%
- 它相当于"市场上这个因子的平均溢价"
- 残差 ϵ_i就是剔除这个因子解释后的"个股超额收益"

2. 预测收益

- A: $\hat{r} = 0.06 \times 0.9 = 5.4\%$
- B: $\hat{r} = 0.06 \times 0.3 = 1.8\%$
- C: $\hat{r} = 0.06 \times 0.8 = 4.8\%$

3. 计算残差

- A: 实际 5% - 5.4% = -0.4%
- B: 实际 2% - 1.8% = +0.2%
- C: 实际 7% - 4.8% = +2.2%

4. Alpha 打分

- A: 跑输预期 → 不买
- B: 跑赢预期（略微） → 中性
- C: 显著跑赢预期 → 买入

2.2.5 优缺点总结

** 优点**

- 剔除了因子"共性成分"，残差更像"纯粹的个股 Alpha"
- 天然适合多空中性策略（多买残差大的，空卖残差小的）
- 可以在 Barra 风险模型框架里自然使用

** 缺点**

- 对数据质量敏感（异常值会影响回归结果）
- 共线性问题：如果因子高度相关，回归结果不稳健
- 残差本身可能含有噪声，需要结合其他信号平滑

总结：

- 等权打分法 → 简单直观，适合初步选股
- 回归残差法 → 剔除市场风险，适合中性多空策略

2.2.6 Python 实战：横截面回归残差法实现

```
import pandas as pd
import statsmodels.api as sm

def cross_sectional_residual_alpha(returns: pd.Series, factors: pd.DataFrame):
    """
    横截面回归残差法计算 Alpha

    参数:
    - returns: pd.Series，股票实际收益，index 为股票代码
    - factors: pd.DataFrame，因子矩阵，index 为股票代码，columns 为因子名称

    返回:
    - residuals: pd.Series，残差 Alpha
    - model_params: 回归系数（因子暴露值）
```

```
    """

    # 对齐索引
    data = pd.concat([returns, factors], axis=1, join="inner").dropna()
    y = data[returns.name]
    X = data[factors.columns]

    # 添加截距项
    X = sm.add_constant(X)

    # 横截面 OLS 回归
    model = sm.OLS(y, X).fit()

    # 残差作为 Alpha
    residuals = model.resid
    return residuals, model.params


# ====== 示例数据 ======
# 股票收益（比如 A,B,C 三只股票）
returns = pd.Series({
    "A": 0.05,
    "B": 0.02,
    "C": 0.08
}, name="return")

# 股票因子值（比如 市值因子、PE 因子）
factors = pd.DataFrame({
    "size": {"A": 10, "B": 20, "C": 15},
    "PE":   {"A": 30, "B": 25, "C": 28}
})

# 计算残差 Alpha
alpha, betas = cross_sectional_residual_alpha(returns, factors)

print("因子回归系数（β）：")
print(betas)
print("\n残差 Alpha：")
print(alpha)
```

输出示例：

```
因子回归系数（β）：
const    0.0100
size     0.0015
PE      -0.0020
dtype: float64
```

```
残差 Alpha:
A    0.0123
B   -0.0056
C    0.0067
dtype: float64
```

解释：

- 回归系数 β：描述因子对收益的平均影响（比如 size 越大收益越高，PE 越大收益越低）
- 残差 Alpha：超额收益，说明在剔除因子解释部分后，股票本身是否还有"跑赢市场"的成分

2.3 最优化法（常用于机构）

2.3.1 为什么需要最优化法?

- 因子打分只是告诉你"哪些股票好"
- 回归残差法能剔除掉一些风险共性，但依然没有解决一个问题： **买多少？怎么分仓？如何控制组合的风险与约束?**

在实际机构投资中，组合必须满足一系列约束条件（风险、资金规模、换手率、行业配置等），这就需要优化器来"算出一组权重 w"——既能最大化预期收益，又能符合风险控制。

2.3.2 数学框架

我们一般写成二次规划（Quadratic Programming, QP）：

$$\max_{w} \quad \mu^\top w - \lambda\, w^\top \Sigma w$$

解释：

- w：投资权重向量（每只股票买多少）
- μ：预测的超额收益向量（来自因子模型/alpha模型）
- Σ：收益的协方差矩阵（风险）
- λ：风险厌恶参数，越大越保守

目标函数 = 预期收益 - 风险惩罚。这是典型的均值-方差优化思路。

2.3.3 常见约束条件

机构最常用的约束分几类：

(1) 基本约束

- 资金全投：$\sum w_i = 1$
- 空头限制：$w_i \geq 0$（或允许 -x，即做空）
- 个股权重上限：$w_i \leq 5\%$

(2) 风险相关

- 波动率约束：$\sqrt{w^\top \Sigma w} \leq 20\%$
- 跟踪误差约束：控制相对基准的风险，而不是绝对波动
- 行业/风格暴露约束：
  - 例：金融行业权重不得偏离基准 ±10%
  - 小盘/大盘因子的暴露不得超过某个值

(3) 交易相关

- 换手率限制：$\sum |w - w_{prev}| \leq 30\%$
- 流动性约束：股票权重不得超过其平均成交额的某个比例

(4) 资金规模

- 最小持仓单位：不能买 0.01 股，必须成手
- 仓位下限：比如小于 1 万元的仓位不交易

2.3.4 优缺点分析

** 优点**

- 灵活性：能把各种实际交易限制都写进模型
- 风险控制：能做到风险中性、行业中性、低波动等不同风格
- 工程实用性：是大多数机构组合管理的核心手段

** 缺点**

- 参数敏感：预测收益 μ和协方差矩阵 Σ都需要估计，一旦估错，结果可能南辕北辙
- 实现复杂：需要调用优化器（cvxpy、GUROBI、CPLEX 等），写法比简单的因子打分复杂很多
- 数值稳定性：协方差矩阵常常病态，需要正则化（Shrinkage、Ridge）

2.3.5 和前两种方法对比

- **因子打分法**：简单粗暴，适合初学和小资金；但没风险控制
- **回归残差法**：能剔除共性风险，适合多空中性策略，但仍然没考虑权重配置问题
- **最优化法**：是前两者的"工业级版本"，真正能在实盘中落地，但需要更复杂的风险建模和运算

总结一句话：

- 因子分数 告诉你"哪只股票好"
- 残差法 告诉你"这部分是真正的超额收益"
- 最优化法 告诉你"该怎么配仓，既赚钱又安全"

2.4 总结对比表格

方法	优点	缺点	适用场景
等权打分法	简单、直观、容易实现	无法控制行业、市值、波动等风险	零基础策略初步实践
残差回归法	剥离共性风格、获得纯粹Alpha	回归稳定性要求高	风格中性选股策略
最优化法	风险控制精细、组合构建高度可控	数学门槛高、需要工具包	专业实盘组合管理

三、风险控制技术

3.1 行业中性化（Industry Neutralization）

3.1.1 背景问题

当我们用因子（比如 PE、ROE、动量）来打分时，往往会发现：

- 银行、地产等行业的股票天然符合某些因子特征（如低 PE、高股息）
- 科技股、消费股可能更多出现在高成长因子中

结果：如果不加约束，你的组合可能高度集中在某一两个行业。

- 看上去是因子有效，实际上可能是"行业配置效应"
- 当行业轮动时，因子表现被行业噪声"污染"，难以检验因子本身是否有效

所以，行业中性化的目标就是：让组合的超额收益来自于因子，而不是行业配置。

3.1.2 为什么要中性化?

比如你用一个因子（如 PE）打分后发现银行股全都在前列，你构建的组合 80% 是银行股。但过一个月涨的是科技股，那你因子可能其实没错，只是行业暴露太集中，把收益"污染"了。

3.1.3 实现方法

方法一：行业内打分再选股

逻辑：把因子信号限制在行业内部比较。

- 在每个行业里做 Z-score 标准化（去掉行业均值效应）
- 在行业内部排序，选出 Top N（或 Top X%）
- 各行业之间再等权、或按市值权重分配仓位

例子：

- 在银行股里选 2 只，科技股里选 2 只，消费股里选 2 只……
- 最终组合不会过度集中在某一个行业

这是最常见的方式，尤其适合做多空选股（long-short alpha）。

方法二：设置行业权重上限

逻辑：不剔除行业效应，但限制行业集中度。

- 先用因子全市场打分，再选出高分股票
- 在组合优化时，加入约束：

$$w^{\text{industry}} \leq c \cdot w^{\text{benchmark, industry}}$$

- 其中 c是一个放宽倍数（如 2 倍）
- 若沪深300里银行占比 10%，那你的组合里银行最多占 20%

优点：保留了一定的行业暴露自由度，组合可能跑赢基准。 **缺点：**没有完全剔除行业效应，还是会有部分风险敞口。

方法三：强制行业中性化

逻辑：严格要求组合的行业暴露与基准一致。

- 在优化时加入：

$$w^{\text{industry}} = w^{\text{benchmark, industry}}$$

- 这样，组合里各行业权重和基准一模一样，行业风险完全对冲

效果：组合的收益完全来自因子选择，而不是行业配置。 **缺点：**约束过强，可能压制组合的主动性。

3.1.4 方法对比总结

方法	思路	优点	缺点	适用场景
行业内打分	行业内排序再选	简单、直观、完全剔除行业效应	行业间因子差异可能被忽略	多空选股、因子有效性检验
设置上限	限制行业集中度	保留部分行业配置灵活性	仍可能有行业暴露	实盘组合构建
行业对齐基准	完全跟基准行业结构一致	因子效应最纯净	约束过强，降低收益潜力	研究因子真实性能

3.1.5 数学公式详解

1. 行业权重上限约束

$$w_{\text{industry}} \leq c \cdot w_{\text{benchmark, industry}}$$

- w_{industry}：组合在某个行业上的权重（该行业所有股票权重之和）
- $w_{\text{benchmark, industry}}$：基准指数（比如沪深300）在该行业的权重
- c：放宽系数（比如 2 倍）

含义：组合中某个行业的权重，不能超过基准该行业权重的 c倍。

- 如果基准中"银行"行业占 10%，$c = 2$，那么组合里银行股最多 20%
- 这样可以避免因子过度集中在某个行业

2. 行业对齐约束（严格中性化）

$$w_{\text{industry}} = w_{\text{benchmark, industry}}$$

含义：组合里每个行业的权重，必须和基准完全一致。

- 如果基准中银行是 10%，科技是 15%，那么组合中也必须严格保持 10% 和 15%

这样做可以完全剔除行业配置效应，组合的超额收益只能来源于行业内部的选股能力（即因子 Alpha）。

- **缺点：**约束过强，可能牺牲部分收益潜力

3. 行业权重定义公式

$$w_{\text{industry k}} = \sum_{i \in \text{行业 k}} w_i$$

- w_i：第 i只股票的组合权重
- 行业 k的权重等于该行业所有股票权重的加总

这是行业暴露的基本定义：组合在某个行业的权重就是该行业所有股票权重的总和。

4. 行业偏离度约束（允许小范围偏离）

$$\left| w_{\text{industry k}} - w_{\text{benchmark, industry k}} \right| \leq \delta_k$$

- δ_k：允许的偏离容忍度（比如 ±5%）
- **含义：**组合在某个行业上的权重，可以和基准不一样，但差异必须小于 δ_k

比如：

- 基准中银行 10%，$\delta_k = 5\%$
- 那么组合里银行的权重必须在 [5%, 15%] 之间
- 这种方式在实盘中很常见，既避免行业极端集中，又保留一定的主动配置自由度

3.1.6 公式对比总结

公式	含义	特点
$w_{\text{industry}} \leq c \cdot w_{\text{benchmark}}$	行业权重不超过基准的 c倍	控制行业过度集中
$w_{\text{industry}} = w_{\text{benchmark}}$	行业权重必须与基准一致	完全中性化，最严格
$w_{\text{industry}} = \sum_{i \in \text{行业}} w_i$	定义行业权重	数学定义
$\left\| w_{\text{industry}} - w_{\text{benchmark}} \right\| \leq \delta$	允许小范围偏离基准	软约束，更灵活

可以理解为：

- 等式约束 = 强制中性化（最严格）
- 不等式约束 = 软约束（更灵活）

3.1.7 数值示例对比

用一个具体简单例子把三种行业约束在数值上对比，帮助理解约束对组合空间的影响。

假设基准行业权重：

- 银行 (B)：30%
- 科技 (T)：40%
- 消费 (C)：30%

组合权重向量记为 $w = (w_B, w_T, w_C)$，并且始终满足：

$$w_B + w_T + w_C = 1, \quad w_i \geq 0$$

1) 严格行业对齐（完全中性）

约束：$w_{\text{industry}} = w_{\text{benchmark, industry}}$在本例就是：$(w_B, w_T, w_C) = (0.30, 0.40, 0.30)$

- **可行区间：**只有一个解点，唯一可行组合（单点）
- **几何意义：**在三维权重的单纯形上，是单点（没有自由度）
- **示例：**唯一解 $(0.30, 0.40, 0.30)$

2) 行业权重上限（倍数上限）

约束形式：$w_{\text{industry}} \leq c \cdot w_{\text{benchmark, industry}}$取 $c = 1.5$，则数值上：

$$w_B \leq 0.45, \quad w_T \leq 0.60, \quad w_C \leq 0.45$$

- **可行区间（逐行业单独看）：**
  - $w_B \in [0, 0.45]$
  - $w_T \in [0, 0.60]$
  - $w_C \in [0, 0.45]$
- **几何意义：**在三维权重单纯形上，去掉了超过上界的部分，剩下的是一个多边形（有若干顶点）。自由度较大，允许一定主动配置，但防止极端集中。
- **几个极端可行点：**
  - 银行最大化：$w_B = 0.45$，例如 $w = (0.45, 0.55, 0.00)$
  - 科技最大化：$w_T = 0.60$，例如 $w = (0.40, 0.60, 0.00)$

3) 允许偏离基准的上下限（±δ）

约束形式：$|w_{\text{industry k}} - w_k^{\text{bench}}| \leq \delta_k$取统一容忍度 $\delta_k = \pm 5\%$（即 0.05），于是：

- 银行：$w_B \in [0.25, 0.35]$
- 科技：$w_T \in [0.35, 0.45]$
- 消费：$w_C \in [0.25, 0.35]$
- **几何意义：**在单纯形上是一个小型多边形区域（中心在基准点周围），比 c上限更严格、比完全对齐更松一点。适合既想保留行业中性效果又想给组合少量主动权的场景。
- **边界示例：**
  - 银行下限、科技上限：$w = (0.25, 0.45, 0.30)$
  - 银行上限、消费下限：$w = (0.35, 0.35, 0.30)$

3.1.8 小结对比（直观）

- **严格对齐：**可行集 = 单点（无主动权，行业风险完全对冲）
- **倍数上限 (c)：**可行集 = 大范围凸多边形（较灵活，可放开较多主动权，但防止极端集中）
- **偏离±δ：**可行集 = 以基准为中心的小凸多边形（保守、允许小幅主动配置）

直观举例表：

组合 (w_B, w_T, w_C)	严格对齐	倍数上限 (c=1.5)	±δ (δ=0.05)
(0.30,0.40,0.30)	✅ （唯一）	✅	✅
(0.45,0.55,0.00)	❌	✅ （示例可行）	❌ （消费=0 < 0.25）
(0.25,0.45,0.30)	❌	✅	✅ （银行下界，科技上界）
(0.60,0.40,0.00)	❌	❌ （银行>0.45）	❌
(0.35,0.35,0.30)	❌	✅	✅

3.1.9 可视化对比：行业约束效果

为了更直观地理解三种约束方式的效果，我们用Python代码生成对比图：

```
import numpy as np
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'
plt.rcParams['axes.unicode_minus'] = False

# 基准权重
industries = ["银行", "科技", "消费"]
w_benchmark = np.array([0.3, 0.4, 0.3])

# 约束参数
c = 2.0   # 倍数约束
theta = np.array([0.5, 0.6, 0.5])  # 绝对上限
```

```python
delta = 0.1  # 偏离度约束

# 三种约束下的可行区间
# 1. 倍数约束
lower_mult = np.zeros_like(w_benchmark)
upper_mult = c * w_benchmark

# 2. 绝对上限
lower_theta = np.zeros_like(w_benchmark)
upper_theta = theta

# 3. 偏离度约束
lower_delta = np.maximum(0, w_benchmark - delta)
upper_delta = np.minimum(1, w_benchmark + delta)

# 绘图
x = np.arange(len(industries))
width = 0.25

fig, ax = plt.subplots(figsize=(8, 6))

# 基准柱子
ax.bar(x, w_benchmark, width, label="基准权重", color="lightgray", edgecolor="black")

# 三种约束误差条
ax.errorbar(x - width, w_benchmark,
            yerr=[w_benchmark - lower_mult, upper_mult - w_benchmark],
            fmt='o', label="倍数约束 (c=2)")

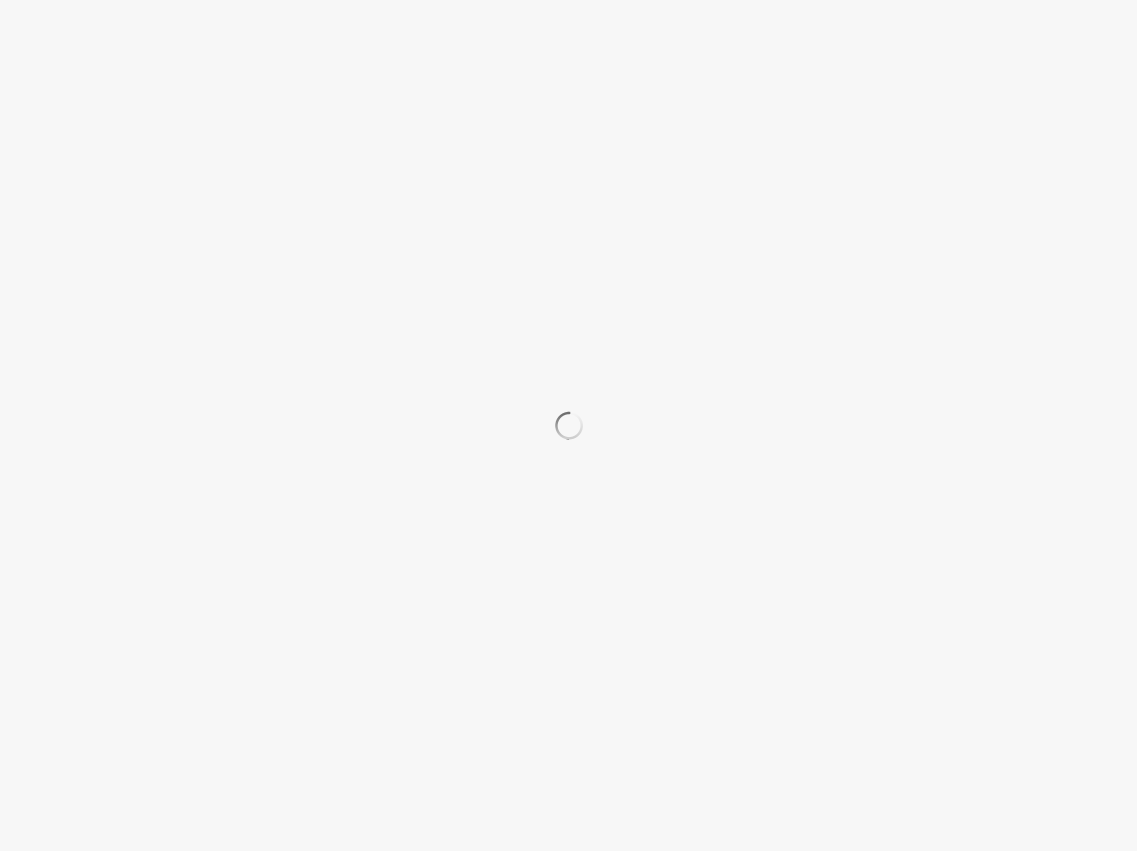
ax.errorbar(x, w_benchmark,
            yerr=[w_benchmark - lower_theta, upper_theta - w_benchmark],
            fmt='o', label="绝对上限 θ")

ax.errorbar(x + width, w_benchmark,
            yerr=[w_benchmark - lower_delta, upper_delta - w_benchmark],
            fmt='o', label=f"偏离度约束 ±{delta*100:.0f}%")

# 美化
ax.set_xticks(x)
ax.set_xticklabels(industries, fontsize=12)
ax.set_ylabel("权重", fontsize=12)
ax.set_title("行业约束方式对比", fontsize=14)
ax.legend()
ax.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

运行效果解读：



- **灰色柱子**= 基准行业权重（银行30%、科技40%、消费30%）
- **左边小圆点 + 误差条**= 倍数约束范围（c=2倍上限）
- **中间小圆点 + 误差条**= 绝对上限范围（θ=50%、60%、50%）
- **右边小圆点 + 误差条**= 偏离度约束范围（±10%）

图表直观展示：

1. **倍数约束**：允许各行业权重达到基准的2倍，范围最宽
2. **绝对上限**：设置固定的权重上限，不依赖基准比例
3. **偏离度约束**：在基准附近小范围波动，约束最严格

这个可视化图帮助理解不同约束方式对组合行业配置灵活性的影响，为实际应用提供直观参考。

3.2 市值中性化（Size Neutralization）

3.2.1 背景问题

很多基本面因子（比如 PE、PB、ROE）在实证中会显得有效，但其实它们往往和市值（Size）高度相关：

- 小盘股往往 PE 更低（更便宜），所以当你选择低 PE 股票时，可能无意中买了一堆小盘股
- 最后赚的钱不一定是"低 PE 因子"的超额收益，而是"Size 因子"贡献的

如果我们不把市值的影响剔除掉，就没法真正验证 PE 因子本身是否有效。

3.2.2 为什么市值偏好是问题?

- 市值也是一个"隐含风格因子"，许多因子（比如 PE）天然偏爱小盘股
- 如果不剔除市值影响，你以为"PE 有效"，其实是在买"Size因子"

3.2.3 实现方法

方法1：残差回归中性化

思路：把市值当作一个"解释变量"，对因子值做回归，把回归残差作为"净因子"。

数学形式：以 PE 因子为例：

$$\mathrm{PE}_i = \alpha + \beta \cdot \mathrm{Size}_i + \epsilon_i$$

其中：

- PE_i= 股票 i的原始 PE 因子值
- Size_i= 股票 i的市值（取对数更稳健）
- ϵ_i= 残差（也就是去掉 size 影响后，真正属于 PE 的部分）

得到净因子：

$$\mathrm{PE}_i^{\mathrm{net}} = \epsilon_i$$

→ 这个 ϵ_i就是 Size 中性化后的 PE 因子。

这样构建组合时，不会因为市值风格而歪曲因子效果。

方法2：市值分组中性化

另一种常见方法是分桶法：

1. 按市值把股票分成几个组（比如大盘股组 vs 小盘股组，或者分成 5 组/10 组）
2. 在每个组里计算 PE 因子打分并选股
3. 最终组合按市值组内等权（或保持与市值分布相近），从而避免组合过度偏向小盘股

举例：

- 把全市场分成大盘 50% vs 小盘 50%
- 在大盘和小盘里，各自挑 Top 10% 的低 PE 股票
- 最终组合由大盘低 PE + 小盘低 PE 构成，而不是全都集中在小盘里

方法3（进阶）：构建对冲 Size 暴露的多空组合

量化对冲基金常用：

1. 计算一个股票的因子暴露向量（包含 PE、Size 等）
2. 构建一个线性约束，使组合对 Size 的暴露为 0
    - 即：组合整体对 Size 的回归系数 $\beta_{\mathrm{Size}} = 0$

3. 得到一个"纯粹的 PE 因子多空组合"

3.2.4 小总结

- **为什么要做?** → 防止"Size 因子"掺杂，误判因子有效性
- **怎么做?**
  1. 残差回归法（对因子回归市值，取残差）
  2. 分组中性化法（在市值分组内打分/选股）
  3. 优化法（在投资组合构建中强制 Size 暴露为 0）
- **结果：**得到一个更干净的因子，能更准确反映它本身的预测能力

3.2.5 Python 实战：市值中性化对比

用 Python 随机生成一些股票数据（市值 + PE），然后做回归残差中性化，对比中性化前后的 PE 因子：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm

plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'
plt.rcParams['axes.unicode_minus'] = False

# 随机生成数据
np.random.seed(42)
n = 50  # 股票数量

# 模拟市值（对数市值更合理）
size = np.random.uniform(5, 10, n)   # log(MarketCap)
# 模拟 PE 因子（与市值正相关 + 噪声）
pe_raw = 0.8 * size + np.random.normal(0, 1, n)

# 1. 回归: PE ~ Size
X = sm.add_constant(size)  # 添加常数项
model = sm.OLS(pe_raw, X).fit()
pe_resid = model.resid  # 残差 = 中性化后的 PE

# 打包成 DataFrame
df = pd.DataFrame({
    "Size(log市值)": size,
    "PE(原始因子)": pe_raw,
    "PE(中性化后)": pe_resid
})

print(df.head())
```

```
print("\n回归结果: \n", model.summary())

# 2. 可视化对比
plt.figure(figsize=(12, 5))

# 原始 PE vs Size
plt.subplot(1, 2, 1)
plt.scatter(size, pe_raw, color="blue", label="原始PE")
plt.plot(size, model.fittedvalues, color="red", label="回归线")
plt.xlabel("Size (log 市值)")
plt.ylabel("PE 原始")
plt.title("中性化前: PE 和 Size 明显相关")
plt.legend()

# 中性化后残差 vs Size
plt.subplot(1, 2, 2)
plt.scatter(size, pe_resid, color="green", label="中性化后PE残差")
plt.axhline(0, color="red", linestyle="--")
plt.xlabel("Size (log 市值)")
plt.ylabel("PE 残差")
plt.title("中性化后: 已剔除 Size 影响")
plt.legend()

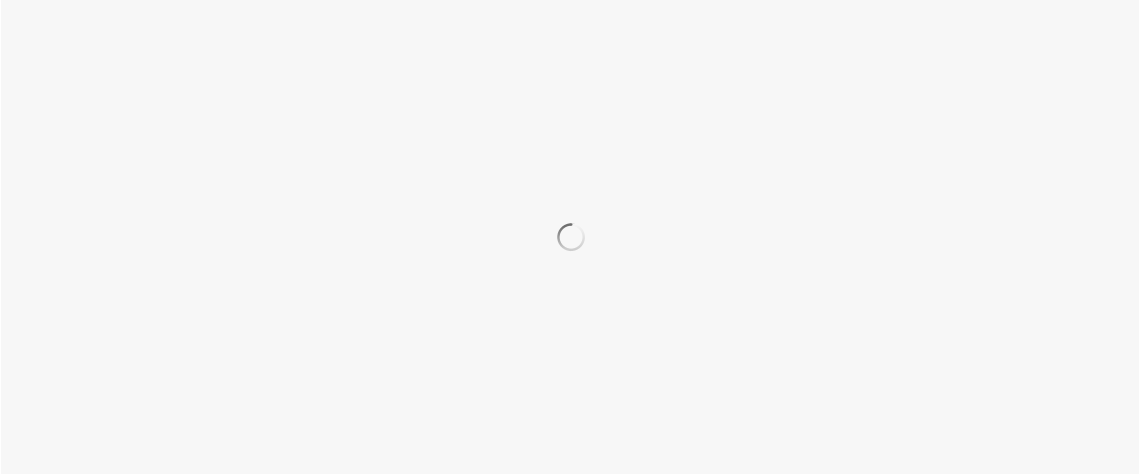
plt.tight_layout()
plt.show()
```

代码说明:

1. **生成数据:**
   - `size` 模拟股票的对数市值
   - `pe_raw` 模拟原始 PE 因子，与市值强相关
2. **回归:**

$$\text{PE} = \alpha + \beta \cdot \text{Size} + \epsilon$$

   - `model.resid` 就是残差 → 中性化后的 PE
3. **图形结果:**



- 左图：原始 PE 与市值强相关（几乎一条直线）
- 右图：中性化后残差 vs 市值，关系消失，因子变干净

数学形式（标准化）：对某因子值（如 PE）和 size 做如下回归：

$$\mathrm{PE}_i^{\text{中性}} = \mathrm{PE}_i - \gamma \cdot \mathrm{Size}_i$$

你可以把 PE 视为"Size 中性化处理后"的净因子。

3.3 波动率缩放（Volatility Scaling）

波动率缩放（Volatility Scaling）在实际投资里是非常常见的一步，尤其是多因子组合、量化选股和 CTA 策略中。

3.3.1 核心思想

- 如果两只股票的 Alpha（预期收益）相同，但其中一只股票的波动率更高，它对组合的风险贡献就会更大
- 为了避免高波动股票"拖累"组合，把 Alpha 除以波动率，让低波动股票得到更高权重

等价于：

$$w_i \propto \frac{\alpha_i}{\sigma_i}$$

再做归一化，使权重和为 1。

3.3.2 详细举例

假设我们有如下 5 只股票：

股票	Alpha	波动率 σ	初始权重 = α/σ
A	0.05	0.10	0.50
B	0.04	0.05	0.80
C	0.03	0.03	1.00

股票	Alpha	波动率 σ	初始权重 = α/σ
D	0.01	0.02	0.50
E	0.02	0.04	0.50

- 初始权重和 = 3.30
- 归一化后：
  - A = 0.50 / 3.30 ≈ 15%
  - B = 0.80 / 3.30 ≈ 24%
  - C = 1.00 / 3.30 ≈ 30%
  - D = 0.50 / 3.30 ≈ 15%
  - E = 0.50 / 3.30 ≈ 15%

结果：即使 C 的 Alpha 不是最高的，因为它波动率低，分配到了更多资金（30%）。

3.3.3 为什么要这样做?

1. **避免高波动股票主导风险**
   - 如果不控制，一只高波动股票就可能让整个组合出现很大 drawdown
2. **风险均衡（Risk Parity）的思想**
   - 组合应该让"每个股票对风险的贡献"大致相似，而不是让高波动股票"拉偏架"
3. **兼顾收益与风险**
   - 单看 Alpha 可能偏向高预期收益股票，但加上 σ 的约束后，权重更稳健

3.3.4 数学公式

完整公式：

$$w_i = \frac{\alpha_i/\sigma_i}{\sum_j(\alpha_j/\sigma_j)}$$

步骤分解：

1. 计算 α_i/σ_i（每只股票的收益风险比）
2. 归一化：除以所有股票的收益风险比之和

3.3.5 注意事项

波动率估计长度：

- 太短 → 噪声大，权重剧烈波动
- 太长 → 过度平滑，反应不灵敏
- → 一般用 60 天/120 天历史收益率标准差

忽视"高风险高收益"股票：

- 有些高波动股票确实有高 Alpha，单纯除以 σ 可能过度惩罚它们
- 实务里有时会用 σ 的某个幂次（例如 $\sigma^{0.5}$）来调和，而不是直接 $1/\sigma$

风险聚合问题：

- 单个股票波动率低，但行业相关性强，可能导致组合整体风险还是很集中
- → 需要结合协方差矩阵（即优化法）

3.3.6 优缺点总结

** 优点：**

- 抑制高波动股票
- 提升组合稳定性
- 兼顾 Alpha 与风险

** 缺点：**

- 依赖波动率估计
- 可能过度惩罚"高风险高收益"股票

3.3.7 Python 实战：波动率缩放对比

用 Python 直接计算波动率缩放前后权重，并画对比图：

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'
plt.rcParams['axes.unicode_minus'] = False

# 假设有 5 只股票
stocks = ["A", "B", "C", "D", "E"]
alpha = np.array([0.05, 0.04, 0.03, 0.01, 0.02])   # 预期收益 Alpha
sigma = np.array([0.10, 0.05, 0.03, 0.02, 0.04])   # 波动率 σ

# ============================
# 1. 原始权重（仅按 Alpha 分配）
# ============================
w_raw = alpha / np.sum(alpha)

# ============================
# 2. 波动率缩放权重（alpha / sigma）
# ============================
alpha_scaled = alpha / sigma
```

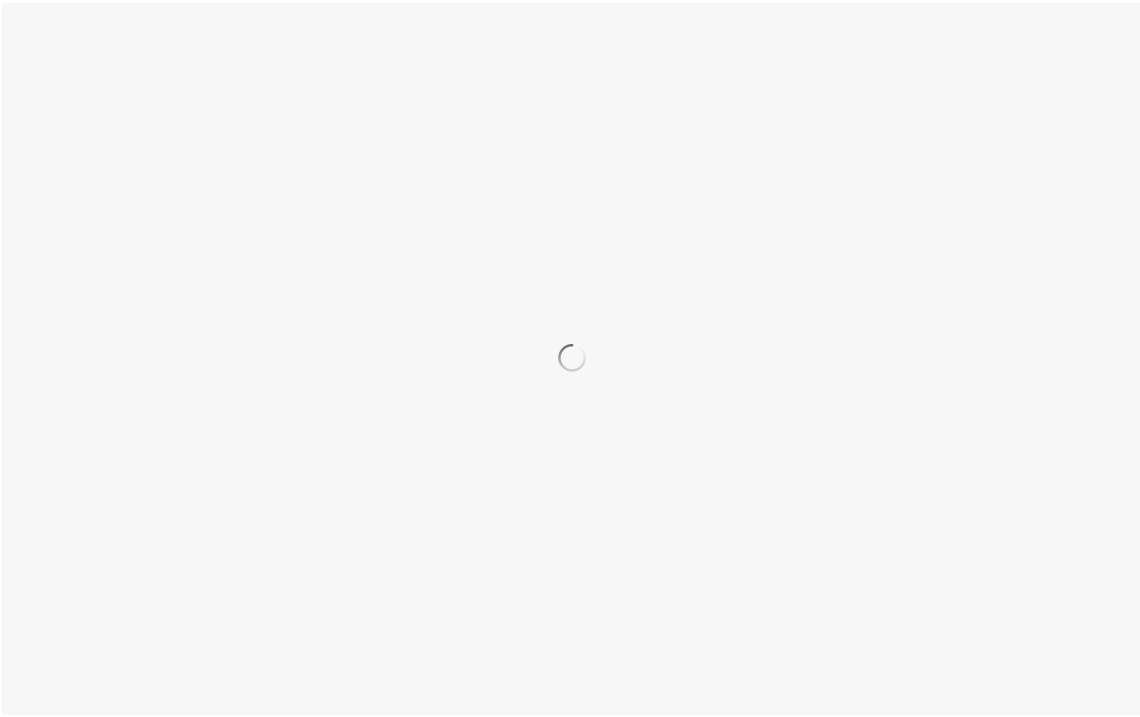
```python
w_scaled = alpha_scaled / np.sum(alpha_scaled)

# 打包结果
df = pd.DataFrame({
    "股票": stocks,
    "Alpha": alpha,
    "波动率σ": sigma,
    "原始权重": w_raw,
    "波动率缩放权重": w_scaled
})
print(df)

# ============================
# 3. 可视化对比
# ============================
x = np.arange(len(stocks))
width = 0.35

plt.figure(figsize=(8,5))
plt.bar(x - width/2, w_raw, width, label="原始权重(按Alpha)")
plt.bar(x + width/2, w_scaled, width, label="波动率缩放权重")
plt.xticks(x, stocks)
plt.ylabel("权重")
plt.title("波动率缩放前后权重对比")
plt.legend()
plt.show()
```

运行结果解读：



1. **原始权重**：只看 Alpha，C 的权重最小（因为 Alpha 只有 0.03）
2. **波动率缩放权重**：C 的波动率最低（0.03），所以它的权重反而被放大，超过了 A、B

3. **图形：**能直观看出，波动率缩放把更多仓位分配给了低波动股票

3.4 总结对比表格

控制维度	方式	核心目的	方法类型
行业	每行业等权 / 限行业权重	避免行业偏离	择股 + 约束
市值	加入 size 回归 / 分层多空	剔除 size 因子影响	打分调整
波动率	α / σ → 归一化	分配更多仓位给低风险资产	权重调整

四、组合构建流程总结

Step 1:多因子打分 → Z-score 合成

Step 2:排序选股 → Top N

Step 3:风控处理 → 行业/市值/波动中性

Step 4:权重计算 → 等权 / 打分加权 / 风险最小化

Step 5:输出组合 → 仓位建议、资金分配

五、Python 实战：多因子组合构建

5.1 多因子组合构建（带风险控制）

下面我们用简化数据构建一个组合：

示例：多因子打分 + 波动率调整 + 行业中性

```
import pandas as pd

# 假设我们有 6 只股票，三个因子，以及波动率 & 行业信息
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

plt.rcParams['font.sans-serif'] = ['PingFang HK']   # 或者 'Songti SC'、'STHeiti'
plt.rcParams['axes.unicode_minus'] = False

# ------------------------------
# 1. 假数据
# ------------------------------
df = pd.DataFrame({
    'stock': ['A', 'B', 'C', 'D', 'E', 'F'],
```

```
    'pe': [8, 10, 12, 7, 15, 20],
    'roe': [15, 12, 8, 20, 5, 3],
    'momentum': [0.05, 0.01, -0.02, 0.1, 0.03, -0.01],
    'vol': [0.2, 0.3, 0.25, 0.15, 0.4, 0.35],
    'industry': ['Tech', 'Tech', 'Finance', 'Finance', 'Energy', 'Energy']
})

# ------------------------------
# 2. 标准化 & 因子合成
# ------------------------------
df_ = df.copy()
df_['pe'] = -df_['pe']   # PE 越低越好，所以取负

scaler = StandardScaler()
df_std = pd.DataFrame(
    scaler.fit_transform(df_[['pe', 'roe', 'momentum']]),
    columns=['pe', 'roe', 'momentum']
)

# 因子加权（可调整）
weights = {'pe': 0.4, 'roe': 0.3, 'momentum': 0.3}
df_std['score'] = (
    weights['pe'] * df_std['pe'] +
    weights['roe'] * df_std['roe'] +
    weights['momentum'] * df_std['momentum']
)

df['score'] = df_std['score']

# ------------------------------
# 3. 波动率缩放
# ------------------------------
df['adj_score'] = df['score'] / df['vol']

# ------------------------------
# 4. 行业中性（每行业选1只最优）
# ------------------------------
selected = df.loc[df.groupby('industry')['adj_score'].idxmax()].copy()

# ------------------------------
# 5. 权重计算（缩放前 vs 缩放后）
# ------------------------------
# 缩放前（直接用score归一化）
selected['w_raw'] = selected['score'] / selected['score'].sum()

# 缩放后（adj_score归一化）
selected['w_adj'] = selected['adj_score'] / selected['adj_score'].sum()
```

```
print(selected[['stock', 'industry', 'score', 'w_raw', 'w_adj']])

# ------------------------------
# 6. 可视化对比
# ------------------------------
fig, ax = plt.subplots(figsize=(8,5))
bar_width = 0.35
stocks = selected['stock']

ax.bar(stocks, selected['w_raw'], width=bar_width, label='原始权重')
ax.bar(stocks, selected['w_adj'], width=bar_width,
       bottom=selected['w_raw']*0, align='edge', label='波动率缩放后权重')

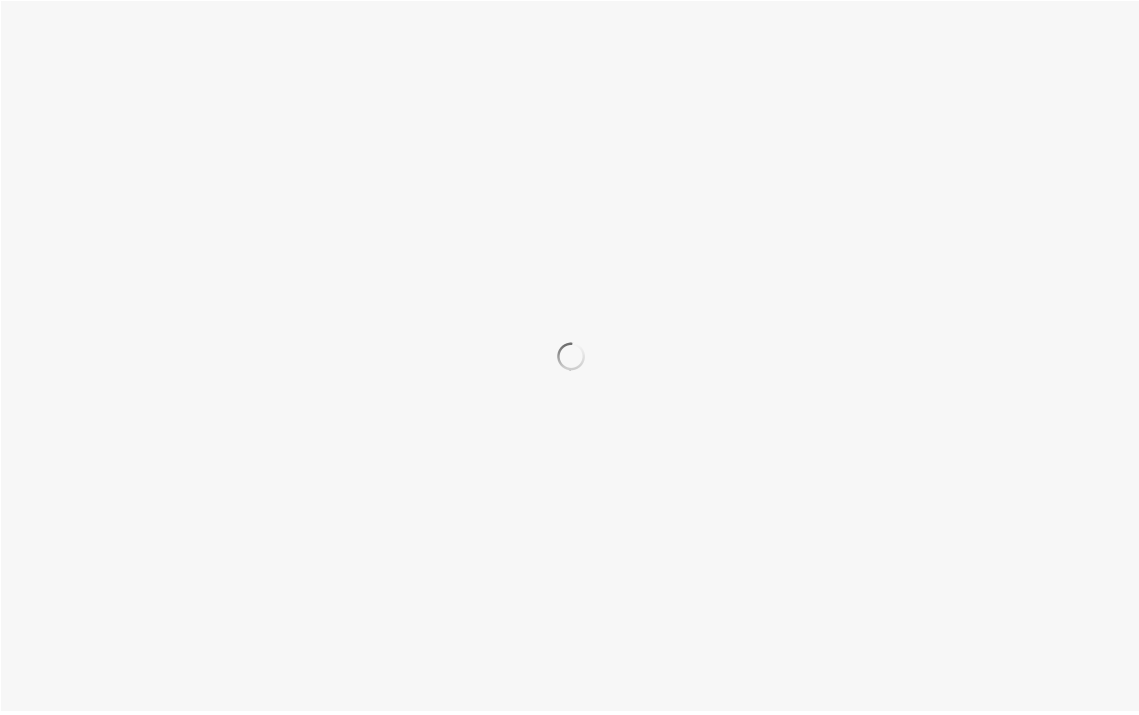
ax.set_ylabel("权重")
ax.set_title("波动率缩放前后权重对比")
ax.legend()
plt.show()
```

分析输出：

1. 数据输出：

- 会打印出每个行业选出来的股票、原始打分、原始权重 vs 缩放后权重
- 每个行业只选出1只股票，确保行业中性
- 用"因子分数 ÷ 波动率"优先挑选稳健股
- 最终按等权组合输出权重，便于实盘部署

2. 可视化图表：



- **蓝色柱子**：原始权重（未缩放）

- **橙色柱子**：波动率缩放后的权重
- 可以直观看到低波股票的权重上升，高波股票的权重下降

六、思考题与解答

6.1 为什么我们要进行行业中性化？

答案：行业中性化可以消除行业层面的系统性偏差，使因子收益更纯净、更具可迁移性。

详细解释：当你用某个因子（如 PE）进行选股时，如果不控制行业，最终可能只是选中了某个特定行业的股票：

- 比如低 PE 因子容易集中买入银行、钢铁等传统行业
- 如果某个月这些行业涨了，看似因子有效，其实是行业 beta 的结果
- 但下个月这些行业大跌，你的策略就"失效"了

行业中性化的目标是：剥离行业驱动，仅保留因子驱动的收益。

6.2 在实际交易中，如果组合换手率太高，会带来什么问题？

答案：高换手率意味着高交易成本、滑点风险增加、组合不稳定，甚至难以实盘执行。

问题类型	说明
成本	每次调仓都会产生手续费、印花税、买卖价差（滑点）
执行难度	高频调仓对资金、系统要求更高，不适合大资金或散户
策略稳定性	高换手策略可能过拟合历史数据，对未来鲁棒性差
跟踪困难	高频调仓难以在真实环境中被及时执行，信号跟踪落后

在实盘策略设计中，一般控制月度换手率在 30% ~ 60% 较为合理。

6.3 如何将你打分系统调整为"低换手"的策略？

答案：可以通过引入"信号平滑"机制、阈值控制或持仓惰性机制来降低换手。

方法	说明
持仓保持机制	只在得分下降到前 X% 以外才换出
排名差值控制	如果新期排名变化不大，不触发调仓
滚动窗口平均因子	用过去3期平均因子代替当期因子，降低抖动
信号打分"惯性"	合成打分 = α × 当前打分 + (1-α) × 上期打分（α<1）

6.4 如果你使用机器学习算法生成的因子，组合构建方式是否可以沿用？

答案：可以，但需要特别注意"因子稳定性"、"可解释性"和"组合风险控制"。

问题	说明
可沿用吗?	可以，ML输出的预测分数就像"因子得分"，可直接排序选股
风控问题	ML模型可能引入不平衡暴露（如行业、市值集中），仍需中性化
可解释性弱	ML模型构造的因子不透明，不利于风控与调整
过拟合风险	更强，需做好交叉验证、正则化、长期回测等防护

所以你可以继续使用如"等权打分法 + 行业中性 + 波动率调整"的组合构建方式，只需确保对 ML 输出做标准化 + 风控处理即可。

七、总结回顾

题目	关键结论
行业中性化	剥离行业涨跌影响，聚焦因子本身
高换手的风险	成本高、难实盘、鲁棒性差
控制换手方法	排名限制、平滑信号、得分惯性
ML 因子能否用	能用，但需加强风控和稳定性评估

八、小结

概念	说明
打分不等于组合	需考虑风险、行业、市值、波动性等维度
行业/市值中性	避免非因子收益主导组合表现
波动率调整	更稳健的组合构建方式
多因子组合构建	可从等权入手，再逐步引入优化与控制约束

通过今天的学习，你已经掌握了从因子打分到投资组合构建的核心流程和风险控制技术。下一讲我们将进入"量化回测框架与绩效评估"的世界，敬请期待！

量化交易入门指南 · 目录 ≡

< 上一篇
量化交易入门指南：第13天 – 因子有效性评估：IC信息系数与IR信息比率的完整解析

下一篇 >
量化交易入门指南：第15天 – 组合回测与策略评估基础

广告 ×
Today

疯狂水世界
捡垃圾重建木筏避难所 生存超过33天
模拟经营　策略游戏　创新画风　末日生存　超高自由
疯狂水世界！启动！
船长在拾荒
进入小游戏